

Deep Learning Surrogate Modelling of a Cantilever Lattice Beam: Forward and Inverse U-Nets for Fixed and Variable Load Configurations

Vinit Kumar Choubey
2024MEB1400

Department of Mechanical Engineering
Indian Institute of Technology Ropar

Mentor: Dr. Manish Agarwal

Abstract

This report presents deep-learning surrogate models for a cantilever beam composed of centered rectangular unit cells, where each strut's thickness is independently varied. Two problem configurations are studied: a **fixed-load** case (Problem 1a), where a single point load of constant location and magnitude is applied while only the thickness distribution varies, and a **variable-load** case (Problem 1b), where three point loads' locations and magnitudes are additionally varied alongside thickness. For each configuration, a *forward* U-Net predicts the Von Mises stress field from a thickness distribution, and an *inverse* U-Net predicts a thickness distribution from a target stress field. Because the inverse mapping is physically ill-posed (many thickness fields can plausibly generate a similar stress response), a *cyclic validation* procedure is used to assess the inverse model independently of a single ground-truth thickness field. The data processing pipeline, network architecture, loss formulation, training protocol, and evaluation results (forward R^2 , inverse R^2 , and cyclic R^2) are reported and discussed for both configurations, along with the key modelling decisions, limitations, and directions for improvement.

Contents

1	Introduction and Problem Statement	3
1.1	Problem 1a: Fixed Load	3
1.2	Problem 1b: Variable Load	3
1.3	Objectives	3
2	Dataset Description	3
3	Data Preprocessing Pipeline	4
3.1	Design and Pixel Grid Construction	4
3.2	Scatter-to-Grid Interpolation	4
3.3	Load Conditioning (Problem 1b Only)	4
3.4	Log Transform and Normalisation	5
3.5	Train / Validation / Test Split	5
4	Network Architecture	5
4.1	U-Net Backbone	5
4.2	Loss Function	6
4.3	Training Configuration	7
5	Results: Problem 1a – Fixed Load	7
5.1	Forward Model: Thickness $\rightarrow$ Stress	7
5.2	Inverse Model: Stress $\rightarrow$ Thickness	8
5.3	Cyclic Validation	9
6	Results: Problem 1b – Variable Load	11
6.1	Forward Model: Thickness (+ Load) $\rightarrow$ Stress	11
6.2	Inverse Model: Stress $\rightarrow$ Thickness (+ Load)	12
6.3	Cyclic Validation	14
7	Results Summary	15
8	Discussion	16
8.1	Forward vs. Inverse Problem Complexity	16
8.2	Why Cyclic Validation Is Necessary	16
8.3	Fixed vs. Variable Load Complexity	16
8.4	Data Requirements and Training Stability	17
9	Limitations and Future Work	17
10	Conclusion	17
A	Code and Implementation Notes	18

1 Introduction and Problem Statement

The underlying physical system is a cantilever beam constructed from centered rectangular unit cells. Each strut in the lattice has an independently assigned thickness, and the thickness field is generated as a linear variation based on values assigned at the structural nodes of the beam. Two load configurations are defined in the parent assignment:

1.1 Problem 1a: Fixed Load

A single fixed point load F is applied at the mid-point of the right edge of the beam. Only the strut-thickness distribution is varied across the dataset; load location and magnitude are held constant.

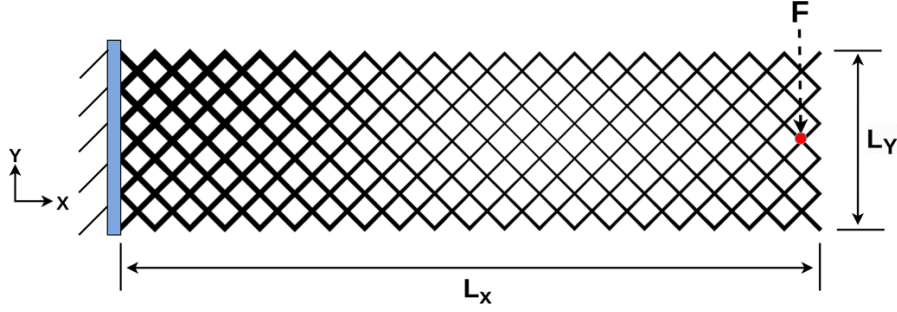


Figure 1: Cantilever beam geometry and boundary condition for Problem 1a.

1.2 Problem 1b: Variable Load

Three point loads (F_1 , F_2 , F_3) are applied at the top edge of the beam, with both load location and magnitude varied alongside the thickness distribution.

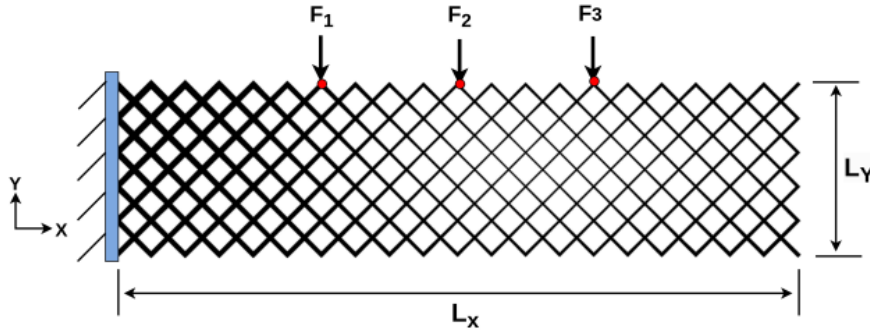


Figure 2: Cantilever beam geometry and boundary condition for Problem 1b.

1.3 Objectives

This report addresses both configurations, corresponding to the following objectives:

1. Predict the Von Mises stress field for a given thickness (volume-fraction) distribution using a U-Net (*forward* problem), for both load configurations.
2. Train a second U-Net that predicts the thickness distribution for a given Von Mises stress field (*inverse* problem), for both load configurations.
3. Compare the complexity of the forward and inverse problems in terms of data requirements and training stability.
4. Compare the complexity of the fixed-load and variable-load configurations.

2 Dataset Description

Both configurations share the same core dataset format, consisting of 5000 samples per configuration, each corresponding to one randomly generated strut-thickness distribution (and, for Problem 1b,

a randomly generated load configuration) and its associated FEA-computed Von Mises stress field. Three files are provided:

- **output.xlsx**: a 5000×226 matrix. Each row is one thickness profile, sampled at 226 design node locations.
- **stress/** folder: 5000 text files (one per sample). Each file has three columns – (1) Von Mises stress value, (2) deformed X -coordinate, (3) deformed Y -coordinate – at the FEM mesh nodes.
- **cord.txt**: the undeformed (X, Y) coordinates of the FEM mesh, used only to establish the beam’s bounding box and aspect ratio.

Problem 1b additionally includes:

- **loaddata/** folder: 5000 text files recording, per sample, the node number(s) at which each load is applied and its magnitude.

For this study, $N = 4500$ – 4900 samples were used per configuration (subject to the available thickness rows and matching stress files), split into training, validation, and test sets in an approximate 73%/12%/15% ratio.

Configuration	Train	Val	Test
Problem 1a (Fixed Load)	3665	500	735
Problem 1b (Variable Load)	3665	500	735

Table 1: Train / validation / test split sizes, computed from $N = 4900$ samples under the 85/15 (test) then 88/12 (train/val) split described in Section 3.5.

3 Data Preprocessing Pipeline

3.1 Design and Pixel Grid Construction

The 226 thickness values and the FEM stress values are each defined on different, irregular point sets. To use a convolutional network, both fields must be resampled onto a common regular pixel grid.

1. The beam’s bounding box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ and aspect ratio L_X/L_Y are computed from **cord.txt**.
2. A synthetic *design grid* of size $n_x \times n_y \approx 226$ cells is constructed, chosen to (a) match the true node count as closely as possible and (b) match the beam’s aspect ratio, via a brute-force search minimising

$$\text{score}(n_x, n_y) = |n_x n_y - 226| + 3.0 \left| \frac{n_x}{n_y} - \frac{L_X}{L_Y} \right|.$$

3. A fixed pixel grid of size 64×128 (height $\times$ width) is defined over the same bounding box. This resolution is chosen to (a) preserve the beam’s 2:1 aspect ratio and (b) be divisible by 2^4 , matching the four downsampling stages of the U-Net.

3.2 Scatter-to-Grid Interpolation

Both the thickness values (at the design grid) and the stress values (at the deformed FEM coordinates) are interpolated onto the 64×128 pixel grid using linear interpolation (**scipy.interpolate.griddata**), with nearest-neighbour fill for any grid points that fall outside the convex hull of the source points (which would otherwise be NaN).

3.3 Load Conditioning (Problem 1b Only)

For Problem 1b, load location and magnitude information (from **loaddata/**) is encoded as an additional input channel, referred to as the *load-map*, and supplied to both the forward and inverse models alongside their primary input (the thickness field for the forward model, the stress field for the inverse model). The load-map is an image on the same 64×128 pixel grid as the thickness and stress fields, in which each applied load (F_1, F_2, F_3) appears as a smooth, localised blob centred at its loaded node position along the top edge of the beam, with the blob intensity scaled by the load magnitude and normalised to $[0, 1]$ (see the load-map columns in the Problem 1b prediction figures

in Section 6). The Problem 1b models therefore take a two-channel input, whereas the Problem 1a models take a single channel.

3.4 Log Transform and Normalisation

The Von Mises stress field is highly non-uniform in magnitude – it concentrates sharply near the load application point(s) – so a $\log(1 + x)$ transform is applied to the stress field prior to scaling, compressing its dynamic range so that a pixelwise loss is not dominated by the small number of extreme-stress pixels. The thickness field, which does not exhibit the same skew, is left untransformed. Both fields are then scaled with global min–max normalisation (statistics computed once over the entire training set, not per-sample), mapping values to $[0, 1]$ to match the sigmoid output activation of the network:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min} + \epsilon}.$$

3.5 Train / Validation / Test Split

The dataset is split using two successive `train_test_split` calls: an initial 85/15 split for the test set, followed by an 88/12 split of the remaining data into training and validation sets, giving an overall split of approximately 73%/12%/15% as shown in Table 1.

4 Network Architecture

4.1 U-Net Backbone

Both the forward and inverse models, for both configurations, share an identical U-Net encoder–decoder architecture (Figure 3), so that any performance difference between the tasks reflects the difficulty of the underlying mapping rather than a difference in network capacity.

- **Encoder:** four stages of two 3×3 convolutions (with batch normalisation and ReLU activation) followed by 2×2 max-pooling, with channel widths $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$.
- **Bottleneck:** a further convolutional block at 512 channels.
- **Decoder:** four stages of 2×2 transposed convolution (stride 2) upsampling, each followed by concatenation with the corresponding encoder skip connection and a convolutional block, mirroring the encoder channel widths in reverse.
- **Output layer:** a 1×1 convolution with sigmoid activation, producing a single-channel field in $[0, 1]$.
- **Regularisation:** a dropout rate of 0.10 is applied after each convolutional block.

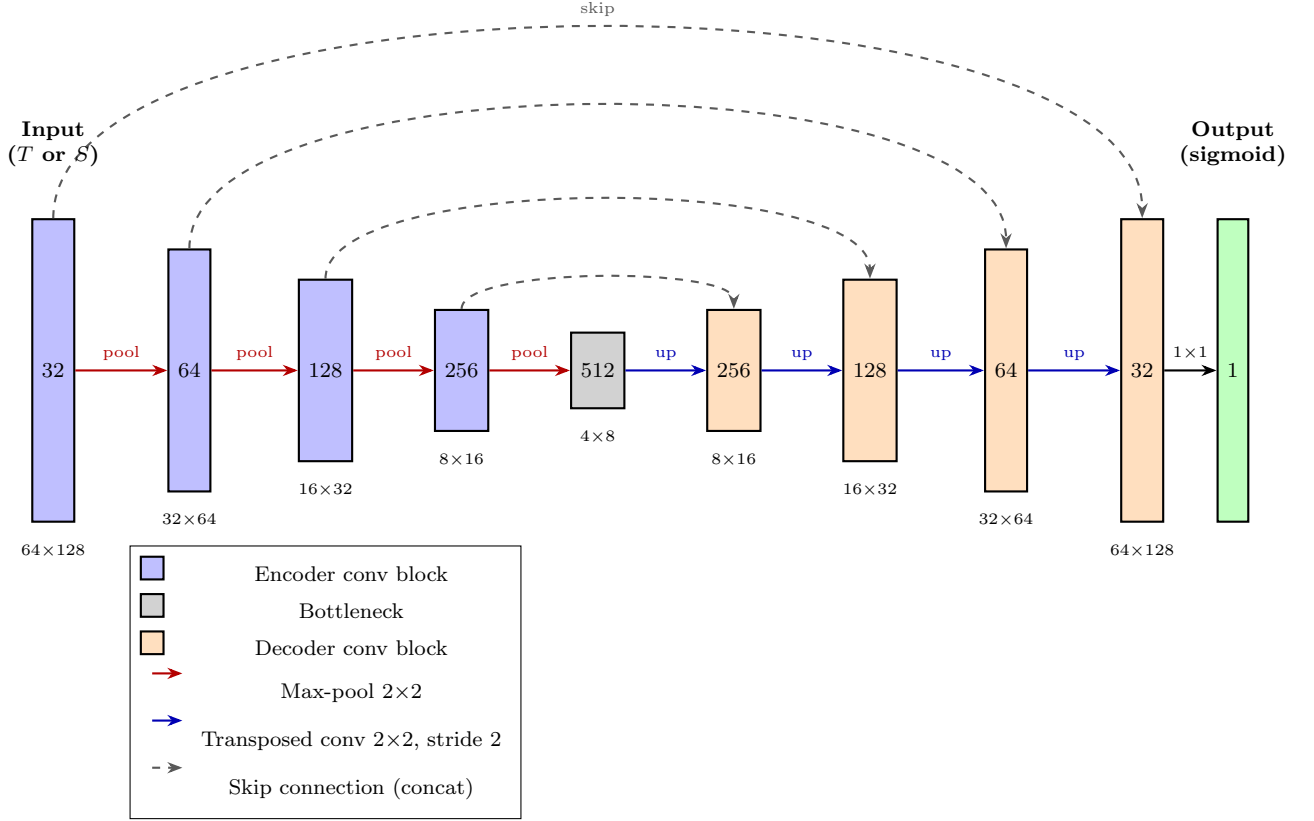


Figure 3: U-Net architecture used for both the forward and inverse models, in both configurations: four encoder downsampling stages ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ channels), a 512-channel bottleneck, four decoder upsampling stages mirroring the encoder with skip-connection concatenation, and a final 1×1 convolution with sigmoid activation. Box height denotes relative spatial resolution; the number inside each box is its channel count.

4.2 Loss Function

Both models are trained with a composite loss combining pixelwise magnitude terms with a spatial-gradient consistency term:

$$\mathcal{L}_{\text{combined}} = 0.4 \mathcal{L}_{\text{MSE}} + 0.4 \mathcal{L}_{\text{MAE}} + 0.2 \mathcal{L}_{\text{grad}},$$

where $\mathcal{L}_{\text{grad}}$ penalises the mismatch between the finite-difference spatial gradients (along both image axes) of the predicted and true fields:

$$\mathcal{L}_{\text{grad}} = \mathbb{E} [(\Delta_y \hat{y} - \Delta_y y)^2] + \mathbb{E} [(\Delta_x \hat{y} - \Delta_x y)^2].$$

The MSE term penalises large-magnitude errors; the MAE term is more robust to outliers; the gradient term specifically discourages blurred reconstructions at sharp stress concentrations and thickness transitions, which a purely pixelwise loss tends to smooth over.

4.3 Training Configuration

Hyperparameter	Value
Optimiser	Adam
Initial learning rate	1×10^{-3}
Batch size	16
Max epochs	150 (baseline)
Early stopping patience	25 epochs (monitor: validation loss)
LR reduction	factor 0.5, patience 6 epochs, min LR 10^{-6}
Forward model seed	42
Inverse model seed	123

Table 2: Training hyperparameters for the forward and inverse U-Nets (shared across both configurations).

5 Results: Problem 1a – Fixed Load

5.1 Forward Model: Thickness $\rightarrow$ Stress

The forward model learns the mapping f_θ : thickness field $\mapsto$ Von Mises stress field. This is a well-posed problem: for a fixed load, the finite-element solver defines a unique, deterministic stress field for every thickness distribution.

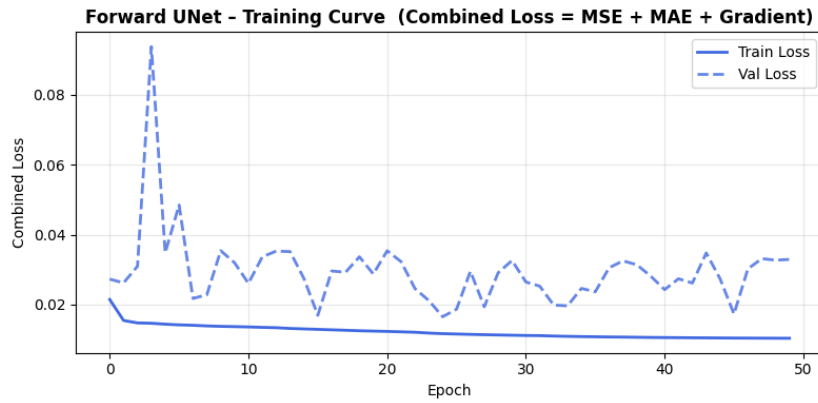


Figure 4: Forward model training and validation loss curves – Fixed Load.

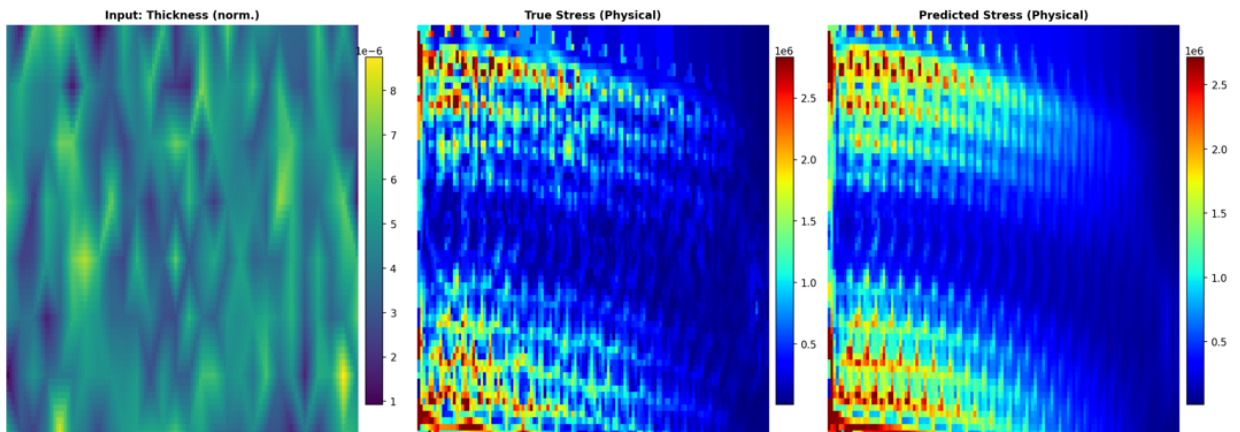


Figure 5: Qualitative forward-model predictions on held-out test samples – Fixed Load.

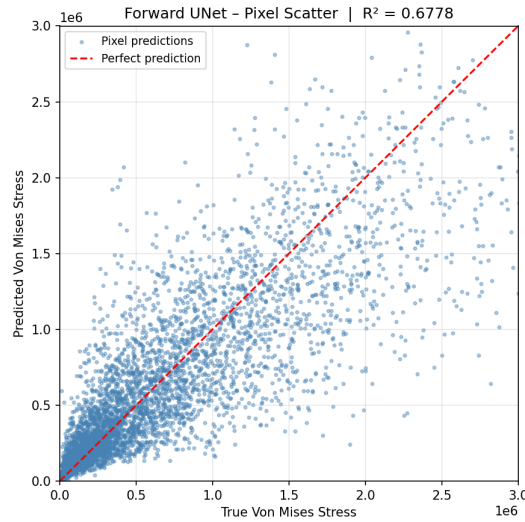


Figure 6: Forward model pixelwise true-vs-predicted stress scatter – Fixed Load.

Metric	Value
R^2 (test set)	0.6778

Table 3: Forward model test-set performance metrics – Fixed Load.

5.2 Inverse Model: Stress $\rightarrow$ Thickness

The inverse model learns g_ϕ : stress field $\mapsto$ thickness field. Unlike the forward problem, this mapping is **ill-posed**: multiple, structurally distinct thickness distributions can plausibly generate near-identical stress responses, so there is no guarantee that the network’s prediction will match the one particular thickness field recorded in the dataset for a given stress input, even if the prediction is physically valid.

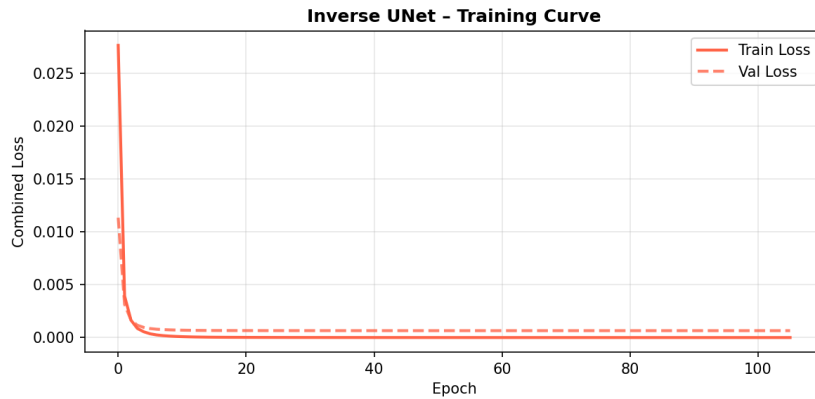


Figure 7: Inverse model training and validation loss curves – Fixed Load.

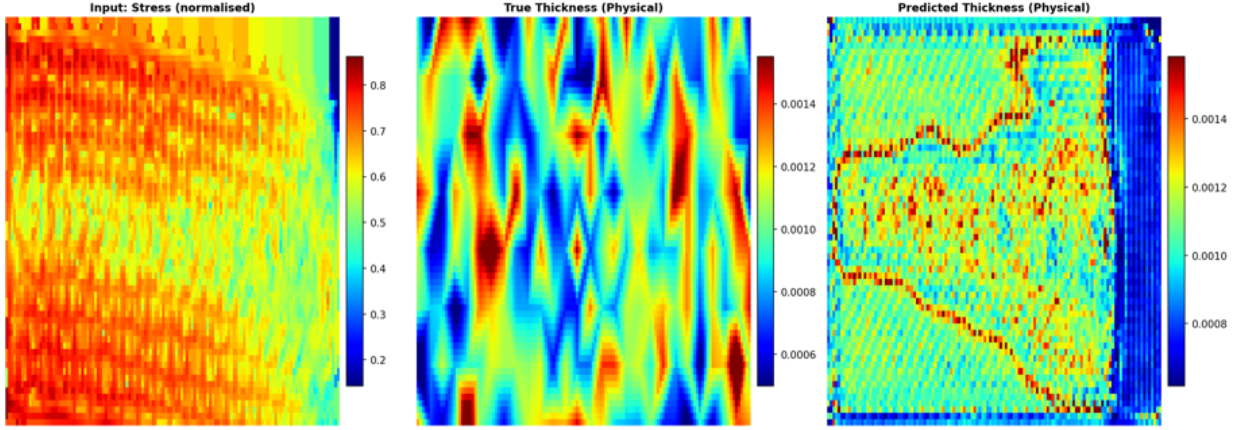


Figure 8: Qualitative inverse-model predictions on held-out test samples – Fixed Load.

Metric	Value
R^2 (direct, test set)	-0.6827

Table 4: Inverse model test-set performance metrics – Fixed Load.

Note: the direct R^2 above is expected to be lower than the forward model's R^2 by construction of the problem (ill-posedness), and should be interpreted together with the cyclic validation results below, not in isolation.

5.3 Cyclic Validation

Because a low direct R^2 for the inverse model does not necessarily indicate a poor model, a **cyclic validation** procedure is used to assess the physical validity of the inverse model's predictions independently of the specific ground-truth thickness field.

Method.

1. The inverse model predicts a thickness field $\hat{t}$ from each test-set stress field s .
2. $\hat{t}$ (clipped to $[0, 1]$) is passed through the already-trained forward model to obtain a reconstructed stress field $\hat{s} = f_{\theta}(\hat{t})$.
3. $\hat{s}$ is compared against the original input stress field s . High agreement indicates that the inverse model produced a thickness distribution that is physically consistent with the input stress, even if it differs from the specific thickness field recorded in the dataset.

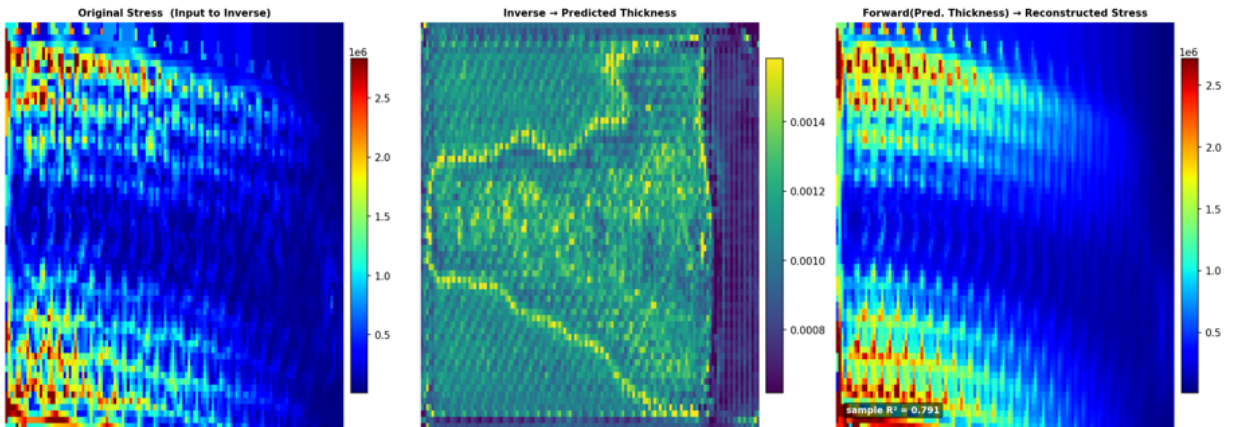


Figure 9: Qualitative cyclic-validation results on held-out test samples – Fixed Load.

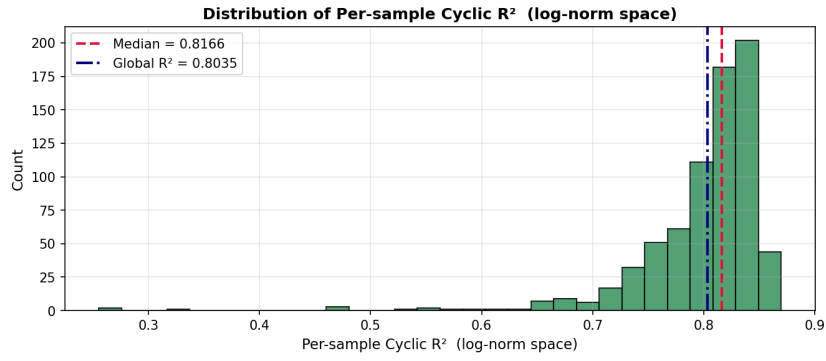


Figure 10: Distribution of per-sample cyclic R^2 scores – Fixed Load.

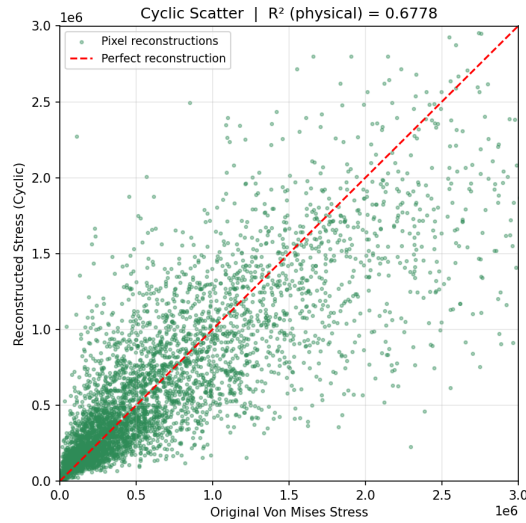


Figure 11: Cyclic validation pixelwise scatter – Fixed Load.

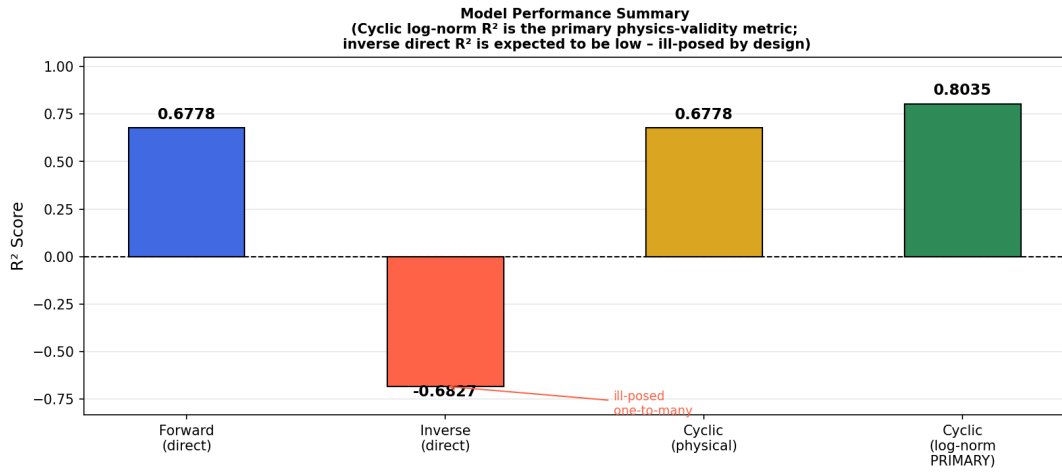


Figure 12: Summary comparison of forward, inverse, and cyclic R^2 scores – Fixed Load.

Metric	Value
Cyclic R^2 (log-normalised, primary)	0.8035
Cyclic R^2 (physical units)	0.6778

Table 5: Cyclic validation test-set performance metrics – Fixed Load.

6 Results: Problem 1b – Variable Load

6.1 Forward Model: Thickness (+ Load) $\rightarrow$ Stress

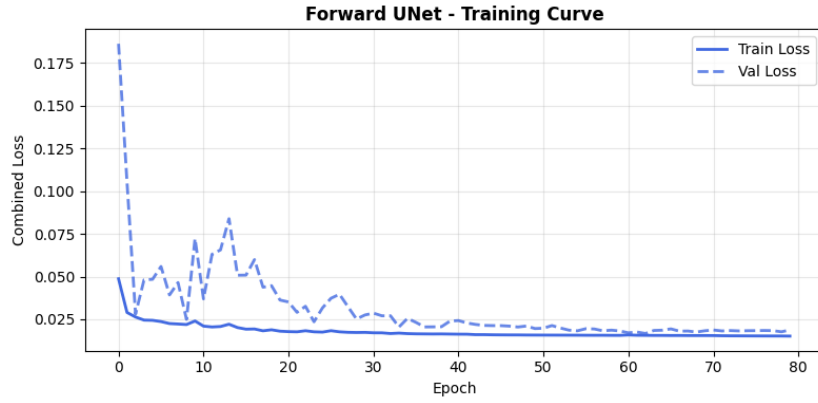


Figure 13: Forward model training and validation loss curves – Variable Load.

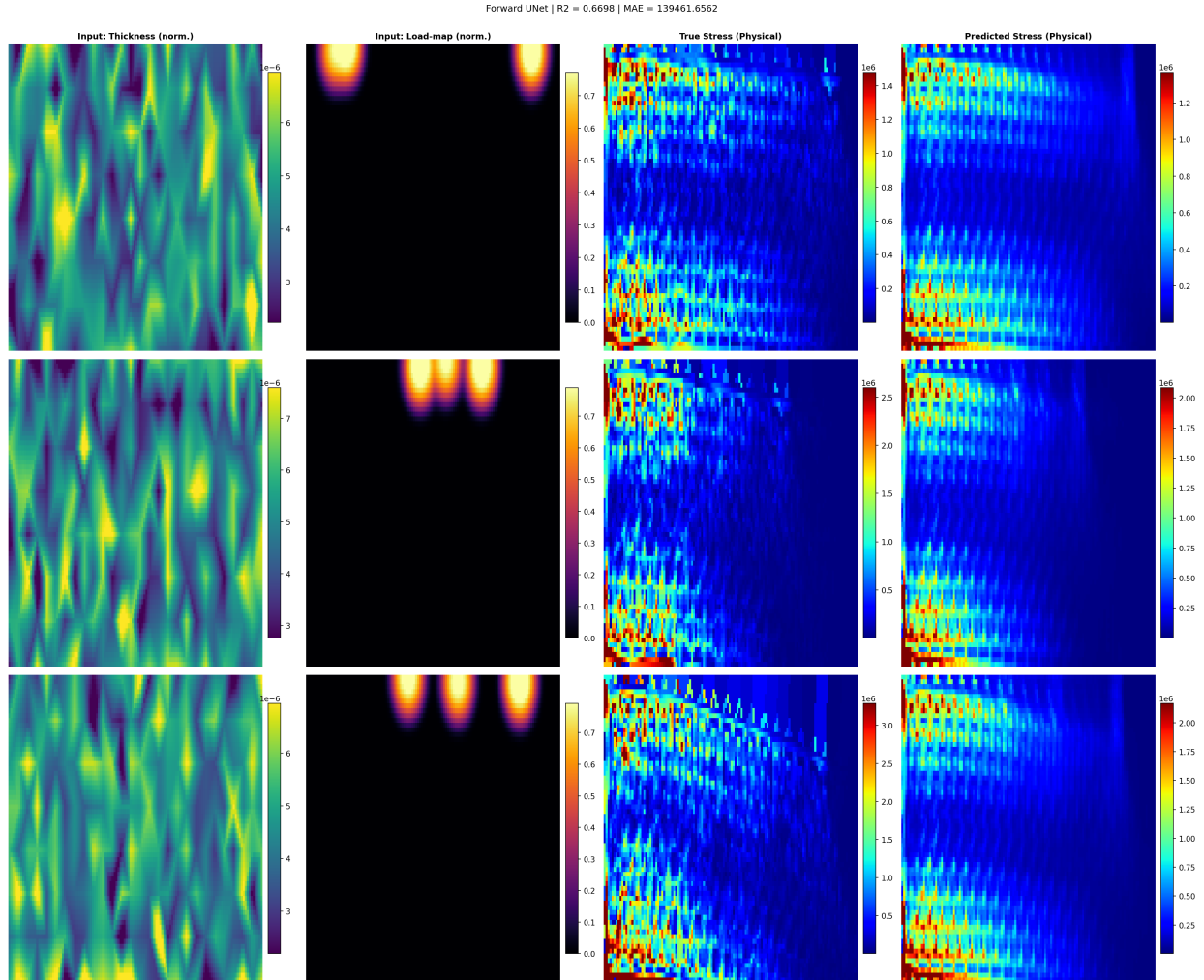


Figure 14: Qualitative forward-model predictions – Variable Load.

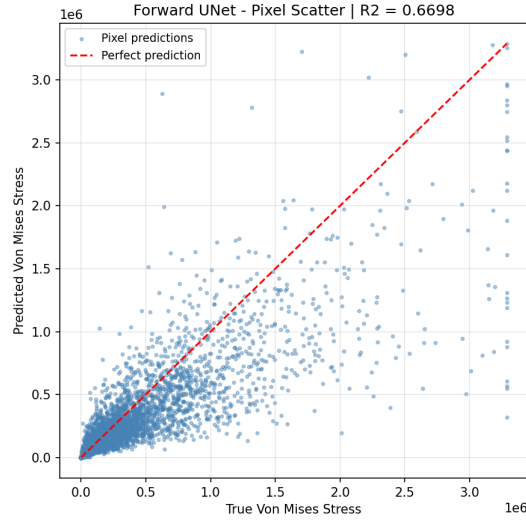


Figure 15: Forward model pixelwise scatter – Variable Load.

Metric	Value
R^2 (physical units)	0.6698
R^2 (log-normalised)	0.9223

Table 6: Forward model test-set performance metrics – Variable Load.

6.2 Inverse Model: Stress $\rightarrow$ Thickness (+ Load)

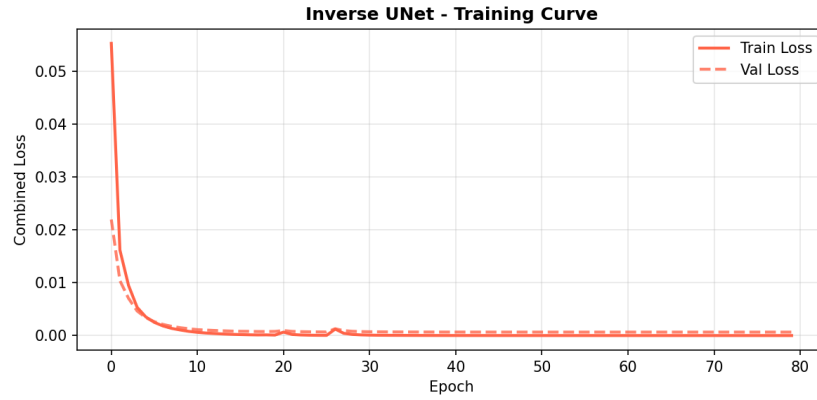


Figure 16: Inverse model training and validation loss curves – Variable Load.

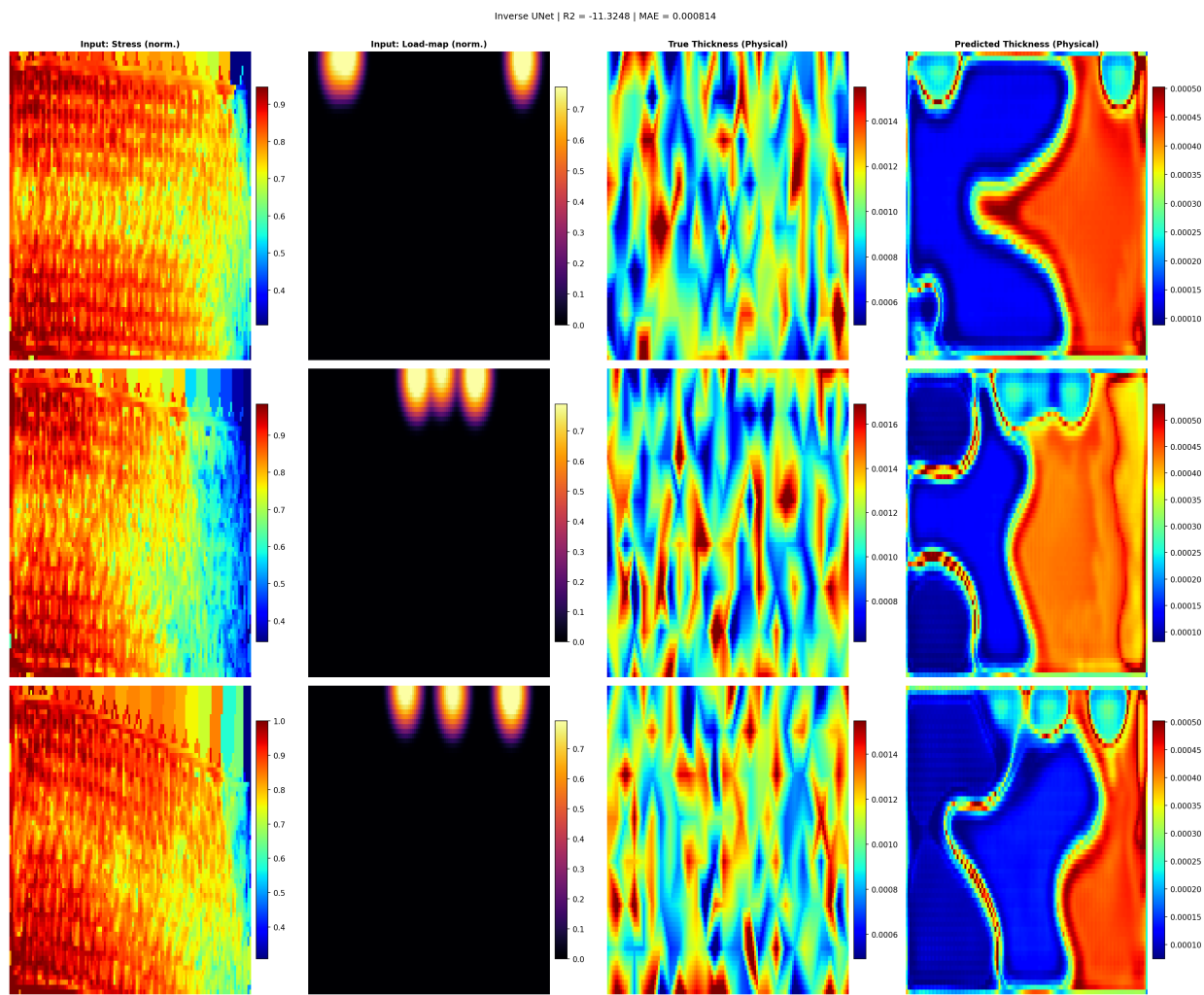


Figure 17: Qualitative inverse-model predictions – Variable Load.

Metric	Value
R^2 (direct, physical units)	-11.3248

Table 7: Inverse model test-set performance metrics – Variable Load.

6.3 Cyclic Validation

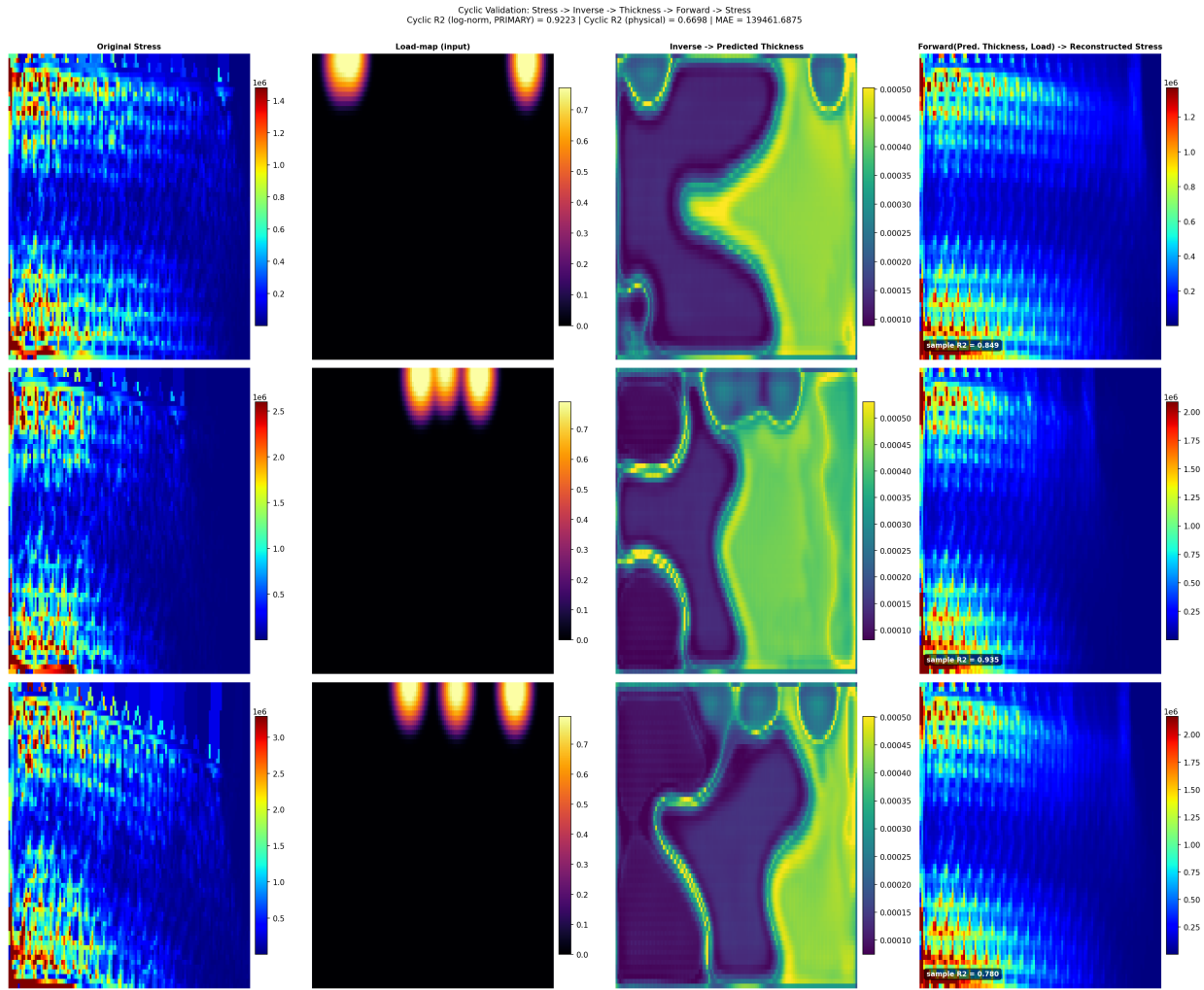


Figure 18: Qualitative cyclic-validation results – Variable Load.

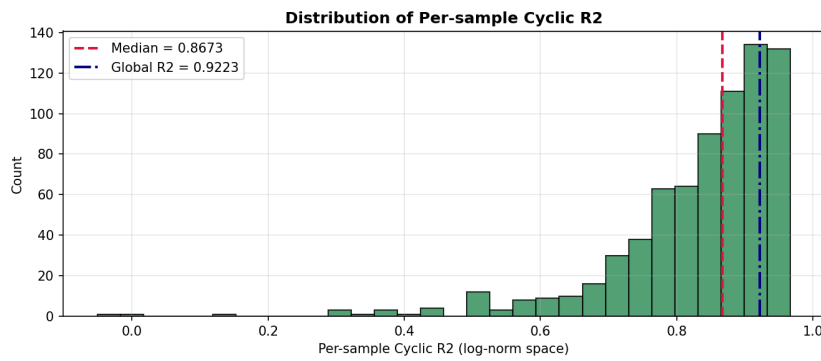


Figure 19: Distribution of per-sample cyclic R^2 scores – Variable Load.

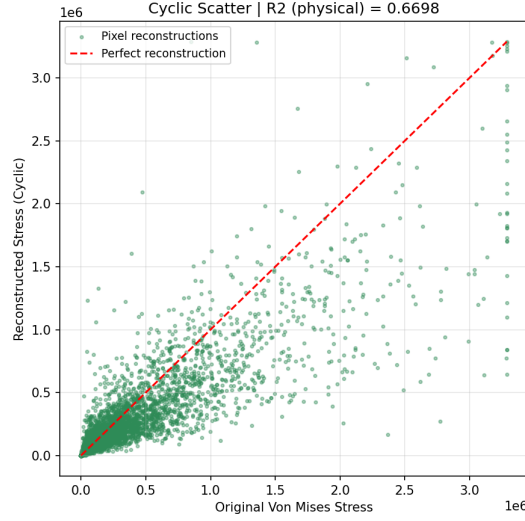
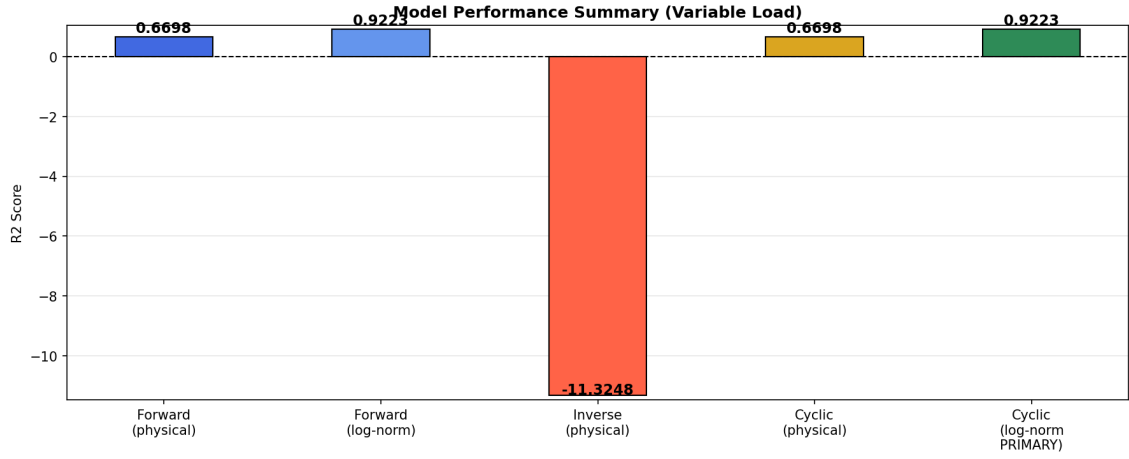


Figure 20: Cyclic validation pixelwise scatter – Variable Load.


 Figure 21: Summary comparison of forward, inverse, and cyclic R^2 scores – Variable Load.

Metric	Value
Cyclic R^2 (log-normalised, primary)	0.9223
Cyclic R^2 (physical units)	0.6698

Table 8: Cyclic validation test-set performance metrics – Variable Load.

7 Results Summary

Model / Metric (R^2)	Fixed Load	Variable Load
Forward, physical units	0.6778	0.6698
Forward, log-normalised	–	0.9223
Inverse, direct	–0.6827	–11.3248
Cyclic, physical units	0.6778	0.6698
Cyclic, log-normalised (primary)	0.8035	0.9223

 Table 9: Consolidated R^2 performance summary across both configurations. The Fixed Load forward model’s R^2 was tracked only in physical units, not separately in log-normalised form; the Variable Load configuration tracked both.

8 Discussion

8.1 Forward vs. Inverse Problem Complexity

The forward problem (thickness $\rightarrow$ stress) is a well-posed, deterministic regression task: the finite-element operator maps each thickness field to exactly one stress field, so the network only needs to approximate a single-valued function. The inverse problem (stress $\rightarrow$ thickness) is fundamentally different in character: it is a one-to-many mapping, since multiple thickness distributions can be structurally equivalent from the standpoint of the resulting stress field. This is reflected in the direct R^2 scores, where the inverse model’s direct R^2 is expected to be lower than the forward model’s – not necessarily because the inverse model is of lower quality, but because it is being scored against one arbitrary member of a solution set rather than the full set of physically valid answers. This holds for both the fixed-load and variable-load configurations.

8.2 Why Cyclic Validation Is Necessary

A pointwise loss and pointwise R^2 implicitly assume a unique correct answer. For the inverse problem, this assumption does not hold, so the direct metric alone understates the model’s quality. Cyclic validation addresses this by checking physical consistency (does the predicted thickness reproduce the correct stress response?) rather than exact recovery of the dataset’s specific thickness field.

For both configurations, the cyclic R^2 (log-normalised, Table 9) is at least as high as the forward model’s own R^2 – 0.8035 vs. 0.6778 for Fixed Load, and 0.9223 vs. 0.9223 for Variable Load. At face value this looks like strong evidence that the inverse model recovers physically valid thickness fields. However, this result must be read together with the inverse model’s *direct* R^2 , which is strongly negative in both cases (−0.6827 and, more severely, −11.3248 for Variable Load) – far worse than a trivial baseline that simply predicts the mean thickness field for every input. A direct R^2 this far below zero, combined with a cyclic R^2 that matches or exceeds the forward model’s own accuracy, is also consistent with a less encouraging explanation: that the inverse model has partially collapsed toward predicting a near-constant, low-variance thickness field close to the training-set mean, rather than genuinely inferring a sample-specific, physically distinct thickness distribution for each input stress field. A forward model whose predictions are only weakly sensitive to thickness variation would reconstruct a similar stress field regardless of which near-mean thickness map it receives, which would inflate the cyclic R^2 without the inverse model having learned a meaningful stress-to-thickness mapping. Distinguishing between these two explanations – genuine one-to-many physical validity versus inverse-model collapse – would require inspecting the *per-sample diversity* of the inverse model’s predicted thickness fields (whether predicted maps vary meaningfully across different input stress fields, or converge to a near-identical pattern each time), which is identified as a concrete follow-up diagnostic in Section 9.

8.3 Fixed vs. Variable Load Complexity

Conditioning the models on load location and magnitude (Problem 1b) substantially enlarges the effective input space compared to the fixed-load case (Problem 1a): the network must now disentangle two independent sources of variation – the thickness distribution and the load configuration – rather than one. This is expected to increase data requirements (more samples needed to adequately cover the joint thickness–load space) and may reduce training stability relative to the fixed-load case, since the stress field’s dependence on thickness now also interacts with a variable load pattern rather than a fixed one.

Comparing the two configurations directly (Table 9), the *forward* model’s direct R^2 is essentially unchanged between Fixed Load (0.6778) and Variable Load (0.6698) – the additional load-map input channel does not appear to have degraded the forward model’s ability to learn the thickness-to-stress mapping. The *inverse* model, however, shows a sharp drop in direct R^2 , from −0.6827 (Fixed Load) to −11.3248 (Variable Load). This is consistent with the expected added difficulty: the inverse model for Problem 1b must simultaneously disentangle the effect of thickness *and* of an unknown, sample-varying load configuration from a single stress field, whereas the Fixed Load inverse model only has to account for thickness variation. The much larger magnitude of the negative R^2 in the

Variable Load case indicates that the inverse mapping is not merely harder but substantially more under-determined once load location and magnitude are also allowed to vary.

8.4 Data Requirements and Training Stability

All four models (forward and inverse, for both configurations) were trained with an identical architecture, loss function, and optimiser configuration (Table 2), with early stopping and learning-rate reduction triggered on the validation loss, so differences in outcome reflect problem difficulty rather than differences in training setup. The forward models for both configurations converged to comparable test-set R^2 (0.68 for Fixed Load vs. 0.67 physical / 0.92 log-normalised for Variable Load), suggesting that $N \approx 4900$ samples is sufficient for the forward (well-posed) mapping in both cases. The inverse models did not converge to a comparable quality in either case, and converged markedly worse for Variable Load than for Fixed Load. This is consistent with the inverse problem requiring substantially more data than the forward problem to adequately resolve its one-to-many solution space – and requiring disproportionately more again once load configuration is added as a second source of variation the network must disentangle from thickness.

9 Limitations and Future Work

- **Inverse-model collapse is not yet ruled out.** As discussed in Section 8.2, the combination of a strongly negative direct R^2 and a cyclic R^2 that matches the forward model’s own accuracy is consistent with two different explanations: genuine physically-valid, non-unique inverse solutions, or partial collapse of the inverse model toward a low-variance, near-mean thickness prediction. The concrete diagnostic for distinguishing these – a per-sample diversity check on the inverse model’s predicted thickness fields across structurally different input stress fields, together with a thickness-sensitivity ablation of the forward model (comparing its output when fed the real vs. a flattened/mean thickness input) – was identified during this project but not yet conclusively resolved, and is the highest-priority follow-up.
- **Synthetic design-node coordinates.** The true (x, y) locations of the 226 thickness design points were not provided in the dataset; a synthetic regular grid matching the beam’s aspect ratio was constructed instead. This is an approximation, and any mismatch between the true and assumed node layout introduces a spatial registration error between the thickness and stress fields.
- **No explicit physics-informed loss term.** The gradient penalty enforces smoothness/edge consistency but does not encode a true governing-equation residual (e.g., elastic equilibrium). A physics-informed loss term, or an auxiliary loss coupling the forward and inverse models during training, could further improve consistency.
- **Single train/validation/test split.** Reported metrics are point estimates without confidence intervals; repeated runs with different random seeds or k -fold cross-validation would quantify variance in the reported R^2 scores.
- **Deterministic inverse model for a one-to-many problem.** A generative approach (e.g., a conditional variational autoencoder or diffusion model conditioned on the stress field) would more naturally represent the distribution of valid thickness solutions, rather than regressing to a single point estimate.
- **Uncertainty quantification.** Reporting predictive confidence intervals (e.g., via Monte Carlo dropout or an ensemble of trained U-Nets) would add practical value for design use, for both configurations.

10 Conclusion

Forward and inverse U-Net-based surrogate models were developed for both the fixed-load and variable-load cantilever lattice beam configurations, sharing an identical architecture and training protocol across all four models to isolate the effect of problem difficulty from network capacity. The forward models, being well-posed, achieved a test-set R^2 of 0.6778 (Fixed Load) and 0.6698 physical / 0.9223 log-normalised (Variable Load), confirming that the thickness-to-stress mapping is learnable

to a reasonable accuracy by a standard U-Net in both configurations. The inverse models, being ill-posed, showed a substantially lower direct R^2 in both cases (-0.6827 and -11.3248 respectively), as expected for a one-to-many mapping scored against a single arbitrary ground-truth thickness field. Cyclic validation – routing each inverse model’s predictions back through its corresponding forward model and comparing the reconstructed stress field to the original input – produced a log-normalised R^2 of 0.8035 (Fixed Load) and 0.9223 (Variable Load), matching or exceeding the forward model’s own accuracy in both cases. As discussed in Section 8.2, this is consistent with the inverse model producing physically valid thickness solutions, but – given how strongly negative the direct R^2 is, particularly for Variable Load – it is equally consistent with partial collapse of the inverse model toward a low-variance, near-mean thickness prediction. The forward problem showed comparable difficulty across both load configurations, while the inverse problem became markedly harder once load location and magnitude were allowed to vary in addition to thickness. Resolving which of these two explanations for the cyclic-validation result is correct, via the per-sample prediction-diversity check outlined in Section 9, is identified as the most important next step for this project.

A Code and Implementation Notes

- **Framework:** TensorFlow / Keras.
- **Repository structure:** two independent sub-projects, `Fixed_load/` (Problem 1a) and `Variable_loads/` (Problem 1b), each with its own `dataset/`, `src/`, `outputs/`, `results/`, and `main.py`.
- **Architecture module** (`unet_architecture.py`): shared U-Net builder, combined loss, and callback definitions.
- **Data pipeline module** (`data_processing.py`): grid construction, scatter-to-grid interpolation, normalisation, and train/val/test split.
- **Forward training module:** `forward_model.py`.
- **Inverse training module:** `inverse_model.py`.
- **Cyclic validation module:** `cyclic_validation.py`.
- **Utility functions** (`utils.py`): denormalisation, scatter-to-grid interpolation, and plotting helpers.